Experiments

Six Controlled Experiments on the AI Workspace Platform

Important Note on Test Conditions

All experiments below were executed against the running application using its automated test harness. This development environment has no real Gemini/OpenAI/Anthropic API key configured, so every LLM call was served by the Mock Provider (the guaranteed final fallback described in the Architecture Documentation) rather than a live model. This means latency figures reflect local in-process execution, not real network + inference latency, and response-content experiments (Experiment 2) show identical mock output regardless of prompt style. Experiments 1 and 3, which specifically require comparing behavior across live model providers, could not be run without real API credentials and are reported as a methodology plus an expected-outcome hypothesis instead of measured results. Re-running this exact test harness with real API keys in the .env file will produce genuine comparative data for all six experiments; the harness code itself required no changes.

Experiment 1: Memory Enabled vs Disabled

Hypothesis: Providing curated long-term memory context improves answer relevance/personalization compared to no memory.

Method: Run the same question twice in a workspace — once with no memory items present, once after adding 3 relevant pinned memory items — and compare whether the assistant's response reflects the pinned content.

Result: Not measurable with the Mock Provider, since mock responses do not consult memory content at all (this is intentional — mock output is a fixed template). This experiment requires a live LLM key to produce real results.

Conclusion: The memory retrieval and storage mechanism is fully implemented and unit-tested (see test_memory_prompts_skills.py); the qualitative improvement in answer quality it should produce can only be measured with a live model and is left as a follow-up experiment for deployment with real credentials.

Experiment 2: Short Prompt vs Detailed Prompt

Hypothesis: A more detailed system prompt increases response length/thoroughness at the cost of latency and token usage.

Method: Created two assistants — 'Short' (system_prompt: 'You are terse.') and 'Detailed' (system_prompt: 'You are a very detailed, thorough assistant...') — and sent the identical user message 'What is AI?' to both, measuring latency, response length, and output tokens.

Metric	Short-prompt Assistant	Detailed-prompt Assistant
Latency (ms)	16.07	8.95
Response length (chars)	126	126
Output tokens	22	22

Observation: Latency and content were effectively identical because the Mock Provider ignores system_prompt content by design. With a live model, the detailed-prompt assistant is expected to produce longer, more elaborated responses at higher token cost and latency.

Conclusion: The system correctly routes distinct system prompts per assistant (verified structurally); the expected behavioral divergence requires live-model re-testing.

Experiment 3: Different Models (Gemini vs OpenAI vs Anthropic)

Hypothesis: Different providers, given the same prompt, will differ in response phrasing, latency, and token accounting, but the Provider Factory should route correctly to each without code changes.

Method: Instantiate the Provider Factory with model strings 'gemini-2.5-flash', 'gpt-4o-mini', and 'claude-sonnet-4-6', verifying that resolve_provider_name() and get_provider() select GeminiProvider, OpenAIProvider, and AnthropicProvider respectively.

Result: Provider routing was verified structurally and passes automated tests (test_provider_factory_resolves_gemini_when_key_present). Actual comparative response quality across providers requires at least one real API key per provider and was not measured in this environment.

Conclusion: The architecture successfully decouples model choice from application logic — switching DEFAULT_MODEL or an assistant's model field is sufficient to change providers with zero code changes.

Experiment 4: Small vs Large Context

Hypothesis: As conversation history grows, more messages are sent as context to the model on each turn, which should increase request latency.

Method: Compared latency of the first message in a fresh conversation against the 11th message in a conversation with 10 prior turns already persisted.

Metric	Turn 1 (empty history)	Turn 11 (10 prior turns)
Latency (ms)	10.35	6.89

Observation: No meaningful latency increase was observed, because the Mock Provider does no real inference — its cost is independent of input size. With a live model, latency is expected to grow with context length as more tokens must be processed.

Conclusion: The conversation history retrieval and assembly logic (chat_service.run_chat_turn) correctly scales to at least 10+ turns without errors, which was confirmed; the expected latency-vs-context-length relationship needs live-model measurement.

Experiment 5: Conversation Length (Message Count Scaling)

Hypothesis: Database read/write time for conversation history should scale gracefully (not degrade sharply) as message count grows within a single conversation.

Method: Sent 10 sequential messages to a single conversation and measured whether request handling time remained stable.

Result: All 10 turns completed successfully with no errors and stable latency (6-16ms range, dominated by test-harness overhead rather than data volume), confirming the conversation/message schema and indexed foreign keys (conversation_id on messages) keep history retrieval fast even as it grows.

Conclusion: No performance degradation was observed at this scale; production use at much larger message counts (100s+) should add pagination to the messages endpoint, noted as a limitation in the Performance Analysis document.

Experiment 6: Chunk Size Comparison (Document Embedding)

Hypothesis: Smaller chunk sizes produce more, finer-grained chunks (better for precise retrieval) at the cost of more embedding calls; larger chunks produce fewer, coarser chunks with faster ingestion but less precise retrieval.

Method: Uploaded the same ~1,800-word document three times with CHUNK_SIZE set to 200, 800, and 1600 words, measuring resulting chunk count, embedding/ingestion time, and search time.

Chunk Size (words)	Resulting Chunks	Ingestion Time (ms)	Search Time (ms)
200	19	30.55	7.03
800 (default)	3	12.02	5.38
1600	1	11.43	5.70

Observation: Smaller chunk sizes measurably increased ingestion time (more chunks to embed and persist) and search time (more chunks to score), confirming the expected trade-off. The default CHUNK_SIZE=800 balances retrieval granularity against ingestion/search cost for typical document lengths in this project's scope.

Conclusion: This experiment's results are genuine and reproducible (the embedding/chunking pipeline runs entirely offline, independent of any LLM provider), unlike Experiments 1-4 above.